

Dynamic Weighted Sparse Matrix Belief Propagation for Polyploid Haplotype Assembly

Thomas Bergendahl

B.S.E., University of Connecticut, 2024

A Dissertation

Submitted in Partial Fulfillment of the

Requirements for the Degree of

Master's in Science

at the

University of Connecticut

2025

Master's of Science Dissertation

Dynamic Weighted Sparse Matrix Belief Propagation for Polyploid Haplotype Assembly

Presented by

Thomas Bergendahl, B.S.E.

Approved by

Major Advisor: Derek Aguiar, PhD

Associate Advisor: Sheida Nabavi, PhD

Associate Advisor: Mukul Bansal, PhD

University of Connecticut

2025

Acknowledgements

I would like to express my deepest gratitude to my advisor, Dr. Derek Aguiar, for his mentorship and support throughout my time spent in his lab. I'd also like to thank the rest of my committee, Dr. Sheida Nabavi and Dr. Mukul Bansal, for their time, effort, and valuable insight on my work. Working with this committee has greatly improved my skills as a researcher and as a communicator.

Many thanks also to all the members of the El GATO lab. Specifically, thank you to Marjan Hosseini, Ella Veiner, and Tala Yassenpoor, who I have worked closely with on the polyploid haplotype assembly problem and the pHapCompass model.

Finally, thank you to my family. They gave me my love for learning, and they were my support system throughout my time as a student, and for that, I will be forever grateful.

Table of Contents

Chapter 1	Introduction	1
1.1	Haplotype Assembly	1
1.2	Polyploid Haplotype Assembly	2
1.3	Belief Propagation	2
1.4	Loopy Belief Propagation	4
Chapter 2	Related Work	6
2.1	Methods for Polyploid Haplotype Assembly	6
2.2	Sparse Matrix Belief Propagation	6
Chapter 3	Motivation	8
3.1	Motivating pHapCompass	8
3.2	Motivating Dynamic Weighted Belief Propagation	8
Chapter 4	Problem Definition	11
Chapter 5	Methods	12
5.1	SNP Graph	12
5.2	pCompass Graph	13
5.3	Weighted Sparse-Matrix Belief Propagation	14
5.4	Computing Node Weights	15
5.5	Sampling Completed Haplotypes	17
Chapter 6	Results	18
6.1	Motivating Example	18
6.2	Simulated NA12878 Data	19
6.3	Quantitative Results on Simulated NA12878 Data	20
6.3.1	Vector Error Rate	20
6.3.2	Pairwise Phasing Accuracy	22
Chapter 7	Discussion	24
Chapter 8	Conclusion	26

Dynamic Weighted Sparse Matrix Belief Propagation for Polyploid Haplotype Assembly

Thomas Bergendahl

University of Connecticut, 2025

Abstract

Resolving haplotype information in genome assemblies is a crucial task which aids in a variety of studies in bioinformatics. Haplotype assembly in polyploid genomes, or samples with more than two distinct haplotypes, remains a difficult problem, due to the ambiguity in assigning sequence reads to a single haplotype and potentially high similarity between haplotypes within subgenomes. These challenges introduce high levels of uncertainty to any possible computed haplotype phasing, and current polyploid haplotype methods do not entertain this uncertainty. I introduce a probabilistic graphical model and a novel dynamic weighted belief propagation algorithm for inference, which uses matrix operations for parallelization. This algorithm preserves uncertainty in the model, allowing for sampling multiple possible haplotype assemblies where it may be appropriate, addressing the additional challenge presented in the polyploid case. I analyzed three variants of this weighted sparse matrix belief propagation algorithm against unweighted sparse matrix belief propagation on simulated polyploid genomes of varying ploidies and varying sequencing read coverage. Analyzing the vector error rate and pairwise phasing accuracy on these results show no considerable difference in performance between weighted sparse matrix belief propagation and unweighted sparse matrix belief propagation.

Chapter 1

Introduction

1.1 Haplotype Assembly

A haplotype is a set of alleles inherited together on the same chromosome in an individual. Haplotype assembly constructs these haplotypes by determining which alleles were inherited together through the use of DNA sequencing reads [13]. There exists an adjacent field, haplotype phasing, which constructs haplotypes through the use of ancestral reference genome data [5]. Haplotype phasing searches for haplotypes which may be shared with the reference genome. Haplotype phasing, without direct observation of sequenced alleles, can be less accurate than haplotype assembly, but produces complete haplotype phasings. Haplotype assembly, on the other hand, often produces incomplete phasings, phasing only in blocks where there is high certainty of phasing in the region.

Haplotype assembly is heavily reliant on the quality and quantity of DNA sequencing reads in order to produce sufficient results. In this work, I focus on sequencing technologies producing short reads. Short-read sequencing replicates the genome many times and divides it into small portions, approximately several hundred base pairs in length. Then, each end of these fragments are sequenced, resulting in paired-end reads roughly 150 base pairs in length.

1.2 Polyploid Haplotype Assembly

Much work has been done concerning diploid haplotype assembly. There are additional challenges, however, that must be addressed when considering polyploid haplotype assembly. In diploid haplotype assembly, homozygous SNP sites are uninformative - both haplotypes will carry the same allele at that site. This also means that haplotypes are complementary. If one haplotype contains the reference allele at a location, the other haplotype will carry the alternate allele. This guarantee of complementarity does not exist, however, in polyploid haplotype assembly. Resolving one haplotype does not necessarily resolve the remaining haplotypes by process of elimination.

The manner in which polyploid genomes develop also contributes difficulty to the haplotype assembly problem. Polyploid genomes are often created through whole-genome duplication [14]. Recent whole-genome duplication events result in highly similar haplotypes within sub-genomes of the polyploid genome. This leads to ambiguity, where there may be long stretches of the chromosome which are completely shared by two or more haplotypes, and only differences outside of the range which a DNA sequencing read would cover. In this case, it is impossible to confidently report a single completely accurate assembly.

1.3 Belief Propagation

Belief propagation [12] is an effective method for inference on probabilistic graphical models. Belief propagation in its classical form is a message-passing algorithm. For this work, I consider belief propagation performed on factor graphs [7]. The algorithm considers the unary and pairwise factors of the graphical model. These factors are similar to probabilistic distributions, where they carry information pertaining to states of random variables in the graph. It also considers marginal 'beliefs', similarly defined on the unary factors of the variables of the distribution. These beliefs are initialized to some specified distribution at the beginning of the algorithm. This initialization can be random, or can be informed by

1.3 BELIEF PROPAGATION

some domain-specific knowledge of the applied problem. Then, the messages of the algorithm are computed, which represent information passed between pairs of dependent nodes. The message updates are computed as such:

$$m_{i \rightarrow j}^{\text{new}}(x_j) \propto \sum_{x_i} \psi_{ij}(x_i, x_j) \psi_{x_i} \prod_{k \in N(i)/j} (m_{k \rightarrow i}^{\text{old}}(x_i)) \quad (1.1)$$

In this formula, $m_{i \rightarrow j}$ is the message from node x_i to node x_j . ψ_{x_i} represents the unary factor for node x_i and $\psi_{i,j}$ represents the pairwise factor between nodes x_i and x_j . $N(i)/j$ denotes all neighboring nodes for x_i , excluding x_j .

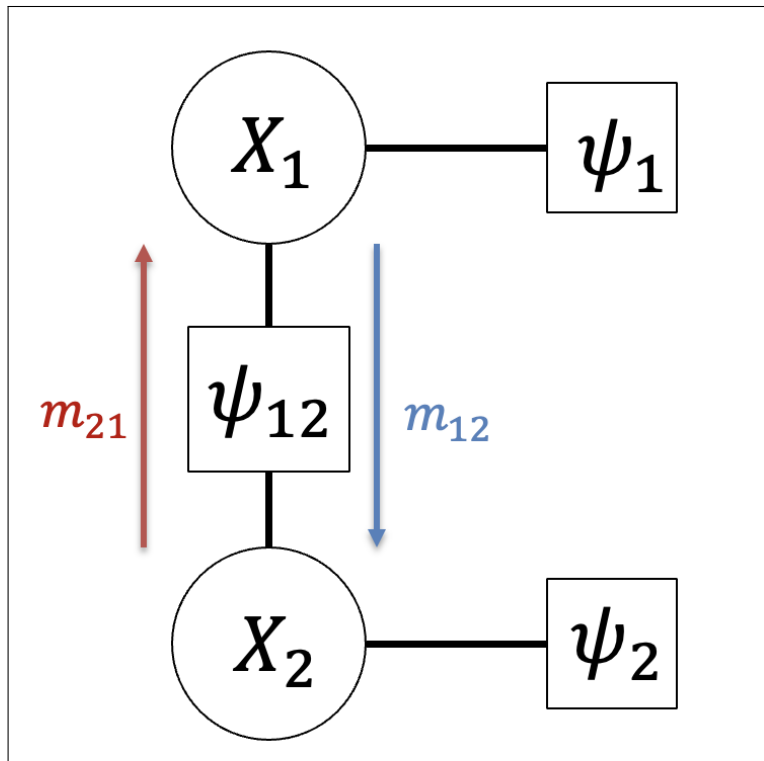
The messages depend on previous messages, as well as the unary and pairwise factors in the initial graph. A message to variable x_j depends on the factors between variable x_j and its neighbors, the unary factors of the neighbors, and the old messages carried by the neighbors. Crucially, there is one message in this region of the graph that is not involved in the computation - the reverse message. Any message from variable x_i to variable x_j is not updated using the message from x_j to x_i .

The node beliefs are subsequently updated using the newly computed message updates, as such:

$$b_i(x_i) \propto \psi_i(x_i) \prod_{k \in N(i)} (m_{k \rightarrow i}(x_i)) \quad (1.2)$$

The belief update for variable x_i depends on the fixed unary factor of variable x_i as well as all messages from the neighbors of x_i to x_i itself.

Once all messages have been computed, and beliefs updated, an iteration has been completed. Message passing (Figure 1.1) iterations are performed until convergence. Depending on the problem, convergence can be measured in a number of ways. Belief propagation is exact in graphs without cycles, due to its feature of excluding the reverse message in the calculation of each message. This means that belief propagation is guaranteed to converge after a finite number of iterations, and when converged, the computed beliefs are equivalent to the true probability distribution. In graphs with cycles, however, belief propagation has no

Figure 1.1: Message passing between two random variables, X_1, X_2

convergence guarantee, and is approximate. Even after the iterations of belief propagation are complete, the computed beliefs may not be equivalent to the true marginals of the distribution. When belief propagation is run on cyclic graphs, it is referred to as 'loopy belief propagation'.

1.4 Loopy Belief Propagation

Loopy belief propagation is not inherently different from classical belief propagation [10]. The same message updates and belief updates are computed, and these computational iterations are performed until convergence. Convergence is achieved differently in loopy belief propagation, however, as loopy belief propagation does not compute the true marginals. Specifically, convergence can be determined when the change in messages between each iteration is minimized. In practice, some threshold is arbitrarily chosen, and when either the distance between the beliefs and the fixed point, or the total change in messages drops below this threshold, convergence is achieved.

1.4 LOOPY BELIEF PROPAGATION

The reason that loopy belief propagation is not exact is that messages can be double counted in graphs with cycles. The double counting phenomenon is avoided in acyclic graphs because the reverse message is not counted in the computation of new messages. However, in cyclic graphs, messages can be counted multiple times. Cycles in the graph can be self-reinforcing, and after multiple iterations, beliefs within these cycles can overpower beliefs in the rest of the graph.

Chapter 2

Related Work

2.1 Methods for Polyploid Haplotype Assembly

H-PopG is a 'heuristic partitioning algorithm', which divides reads into k groups, where k is equal to the ploidy of the genome, such that the differences or incompatibilities between reads within these groups is minimized [17]. Similarly, WhatsHap performs read clustering, and assembles k haplotypes from k clusters [11].XHap and DeepHapNet are recent methods which leverage deep neural networks to solve the haplotype assembly problem [4] [9]. XHap uses transformers to discover long range dependencies between reads which may not necessarily overlap. DeepHapNet performs a deep spectral clustering operation, which uses the neural network to cluster similar reads, similar to existing, non-deep, methods.

2.2 Sparse Matrix Belief Propagation

Sparse-matrix belief propagation [3] is a practical improvement on the belief propagation algorithm which aims to provide significant speed-ups to the implementation of the algorithm. In classical belief propagation, message and belief updates are computed sequentially. Due to this, in large graphs, belief propagation can be computationally expensive. Sparse-matrix belief propagation instead proposes to compute these message and belief updates in parallel.

2.2 SPARSE MATRIX BELIEF PROPAGATION

The updates are encoded as matrix operations, such that they can be computed efficiently with multiple processing cores. First, the belief updates are calculated, with the following formulae:

$$B' = \Phi + M^T T \quad (2.1)$$

$$B = B' - \text{LogSumExp}(B') \quad (2.2)$$

In these updates, B represents the matrix containing the beliefs of each node. B has dimensions $c \times n$, where c is the maximum number of states for any variable and n is the number of variables. Φ is the matrix containing the fixed unary factors of each variable, with the same dimensions as B . M is a matrix containing messages, with dimensions $c \times |E|$. T is a matrix which maps these messages to the proper variables, with dimensions $|E| \times n$. After the belief matrix update is computed, the message updates are computed.

$$M = \text{LogSumExp}(\Gamma + B[:, s] - M[:, r]) \quad (2.3)$$

In this formula, Γ is a matrix containing the pairwise potentials of the initial factor graph, with dimensions $c \times c \times |E|$. B and M are the same matrices as contained in the belief update. The slice notation in $B[:, s]$ and $M[:, r]$ denotes that each message in the updated message matrix is computed using the incoming messages of the variable's neighbors, exactly as it's done in classical belief propagation.

Sparse matrix belief propagation showed considerable improvements in computational speed [3]. Several implementations of classical belief propagation were compared to sparse matrix belief propagation, both operated on CPU and on GPU. The parallelized GPU implementation performed at least twice as fast as the other implementations, especially as the number of variables in their tests increased. These tests were all performed on randomly simulated datasets of grid-like structure, with varying cardinality of factors.

Chapter 3

Motivation

3.1 Motivating pHapCompass

Considering the unique challenges in polyploid haplotype assembly and shortcomings of current methods, we propose a probabilistic graphical model for representing pairwise phasings between SNPs called pHapCompass. We build this probabilistic model with the intent of addressing these primary objectives:

- We aim to build a model which can accept any relevant haplotype evidence, rather than restrict the input space to sequencing reads.
- This model should maintain and quantify uncertainty over many possible complete haplotype phasings.
- The output for this method should be complete haplotype phasings instead of incomplete phasing blocks.

3.2 Motivating Dynamic Weighted Belief Propagation

Modeling the haplotype assembly problem with a probabilistic approach requires a probabilistic inference method to reach a final solution. I first considered belief propagation for

3.2 MOTIVATING DYNAMIC WEIGHTED BELIEF PROPAGATION

inference, but belief propagation may have a significant drawback when applied to haplotype assembly. Consider the example shown in Figure 3.1, where four random variables are represented in a factor graph, each with a unary factor representing a distribution over two states.

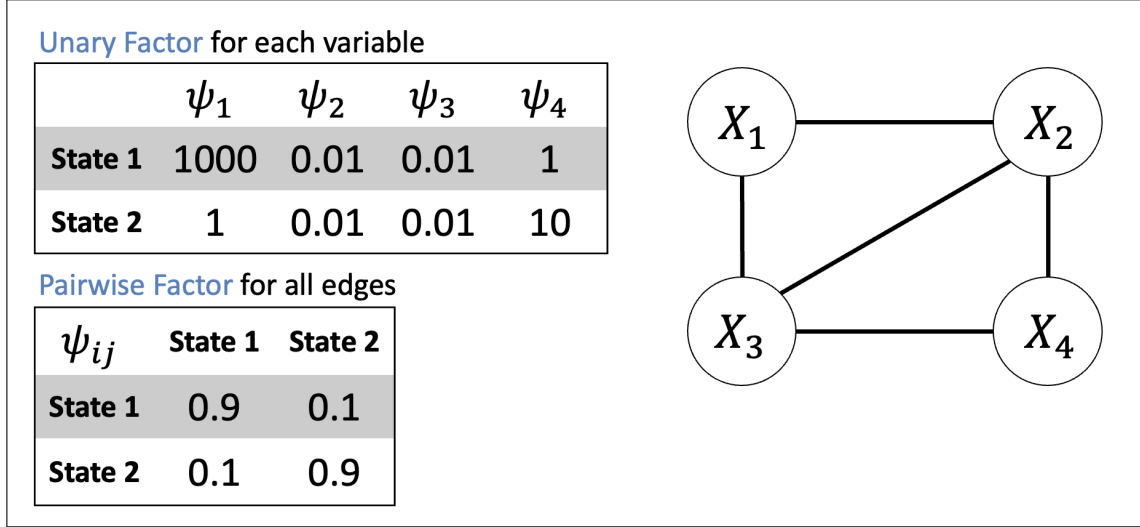


Figure 3.1: Consider this factor graph with four binary random variables. Unary factors for each variable, and the pairwise factor for all edge factors are shown in the tables on the left.

In this example, the unary factor for variable X_1 contains a high factor mass for state 1 and low factor mass for state 2. The factors for variables X_2, X_3 carry little factor mass overall, but evenly distributed over the states. The factor for X_4 contains little factor mass for state 1 and slightly more factor mass on state 2. The pairwise factor for all edges in the factor graph encourage neighboring nodes to take the same state.

Now consider the marginal probabilities of this same factor graph after executing belief propagation (Figure 3.2).

The marginal probabilities reveal that X_1 is always in state 1, X_2 and X_3 have equal probability for either state, and X_4 has 0.99 probability for state 2. These results may be desirable in applications for classical belief propagation. However, if we apply this example to the haplotype assembly problem, we may expect different behavior.

Consider each random variable to represent the phasing of a location on the chromosome

3.2 MOTIVATING DYNAMIC WEIGHTED BELIEF PROPAGATION

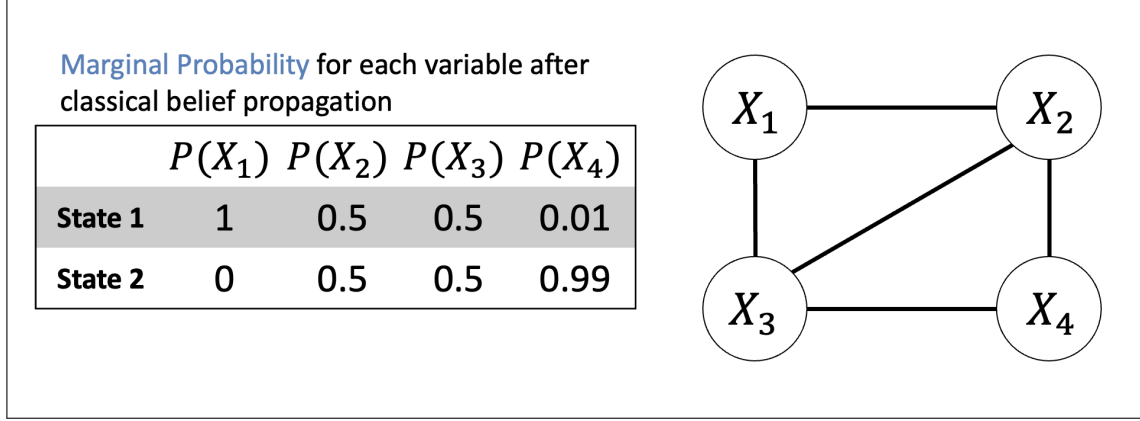


Figure 3.2: Marginal probabilities for each random variable can be seen on the left, after belief propagation has been completed on the factor graph.

in the haplotype assembly problem. Also, let the size of each factor be determined by the amount of read evidence we have covering each location on the chromosome. In this case, X_1 is the location which has the most read evidence, and this may suggest that the phasing for X_1 is the most informative of the complete haplotype phasing. The phasing at X_1 should, in turn, heavily influence the phasings at the other nodes in the graph. However, this is not the case in classical belief propagation. Each message is weighted equally in belief propagation, and the information in the most informative nodes is not spread through the graph. If this was the case, then more probability mass would be on state 1 for X_2, X_3, X_4 . In order to achieve the desired behavior, I propose a weighted approach to belief propagation.

Chapter 4

Problem Definition

The input to the haplotype assembly problem is a set of sequence reads, $R = \{r_1, r_2, \dots, r_n\}$ containing the positions and alleles of single nucleotide polymorphisms (SNPs) which have been aligned to a reference genome, the ploidy of the organism, k , and the genotype information of the organism, $G = \{g_1, g_2, \dots, g_n\}$ where $g_i \in \{1, \dots, k-1\}$, such that g_i represents the amount of haplotypes which carry the alternative allele at position i in this organism's DNA. The objective of the haplotype assembly problem is to produce a haplotype-resolved assembly which is most likely, given the input sequencing reads and genotype information.

Chapter 5

Methods

5.1 SNP Graph

Our method first constructs the SNP graph. The SNP graph, $G_s(V_s, E_s)$ is defined as follows: the vertex set, V_s , contains a vertex for every SNP found in the input DNA sequencing reads, and the edges in edge set E_s connect the nodes containing SNPs which appear on the same DNA sequencing read (Figure 5.1). This graph is constructed as an intermediate step simply to make the construction of the next graphical model more efficient.

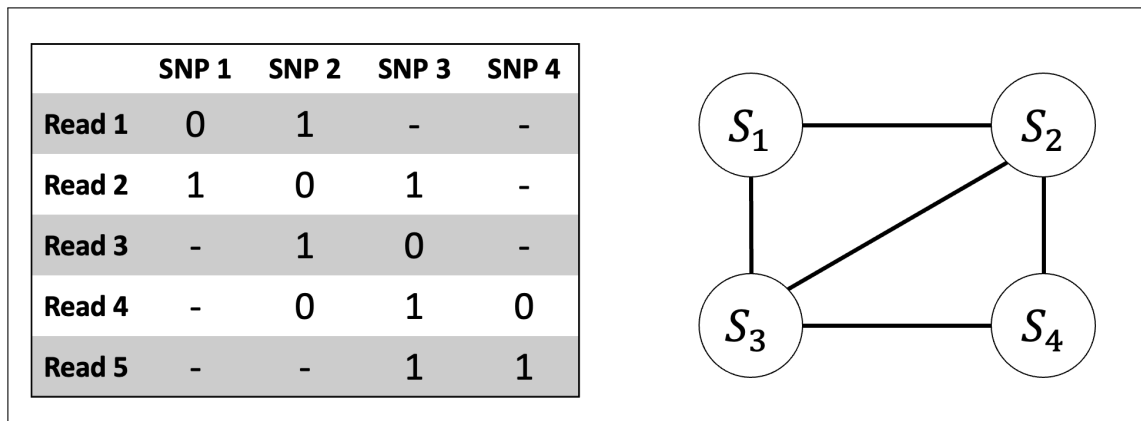


Figure 5.1: Example SNP graph construction. The nodes of this graph represent SNPs in the sample, and the edges connect SNPs which share a DNA read fragment.

5.2 pCompass Graph

Using the SNP graph, we then build the pCompass graph, which is a factor graph encoding the pairwise phasings between SNPs (Figure 5.2). The pCompass graph $G(V_p, E_p)$ is defined as follows: we construct a vertex for every pair of SNPs which appear together on a DNA sequencing read, and construct an edge between vertices which share a SNP position. This graph can be constructed directly from the SNP graph in the prior step of the algorithm. The edge set, E_s , of the SNP graph becomes the vertex set, V_p of the pCompass graph, such that an edge connecting nodes S_1 and S_2 in the SNP graph becomes a vertex $v_{1,2}$ in the pCompass graph.

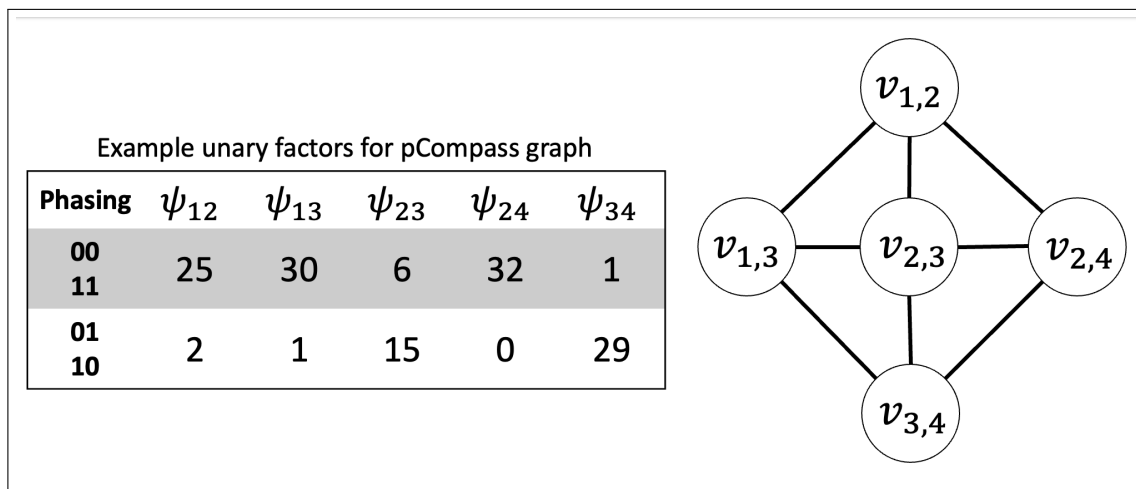


Figure 5.2: pCompass graph construction

The factors on this graph are defined on the nodes and the edges. The node factors represent the distribution over possible phasings between the two SNPs represented at the node. As such, the size of the node factors are equal to the number of possible phasings of the two SNPs. This quantity is dependent on the ploidy and the genotype of the genome. These node factors are determined by the amount of DNA read evidence that supports each pairwise SNP phasing.

5.3 Weighted Sparse-Matrix Belief Propagation

This novel method for inference on the pCompass factor graph introduces a dynamic weighting procedure to belief propagation, and operates on the sparse-matrix belief propagation platform. Each node in the graph is assigned a weight, and the messages sent from that node are multiplied in value by the node's weight. Node weights are updated proportional to the average of the weight of all incoming messages, as well as the node's own weight.

First, the unary, Φ , and pairwise, Γ , potential matrices are set to the initial factors in the pCompass graph. These matrices are fixed, and will not change during the belief propagation process. The variable belief matrix, B , is initialized to the unary potential matrix, as is seen in the first iteration of unweighted sparse belief matrix propagation (as the message matrix is empty in this step).

$$B' = \Phi \tag{5.1}$$

$$B = B' - \text{LogSumExp}(B') \tag{5.2}$$

Similarly, the message matrix, M , is computed exactly the same as in unweighted sparse matrix belief propagation. The initial messages are a log sum of the pairwise belief matrix, Γ , and the initial computed belief matrix, B , which is sliced by s , representing the slices for the variable each message is sent from.

$$M = \text{LogSumExp}(\Gamma + B[:, s]) \tag{5.3}$$

Then, in subsequent iterations of the algorithm, the weighting process is applied. The variable beliefs are updated as the sum of the unary potential matrix and the product of the transposed message matrix and the weighted mapping matrix, $w(T)$. This matrix is obtained by multiplying each 1 in the original mapping matrix, which designates the variable which the message is being sent to, by the weight of the node that the message is sent from.

$$B' = \Phi + M^T w(T) \quad (5.4)$$

$$B = B' - \text{LogSumExp}(B') \quad (5.5)$$

The new message matrix is computed in exactly the same way as unweighted belief propagation, as seen below.

$$M = \text{LogSumExp}(\Gamma + B[:, s] - M[:, r]) \quad (5.6)$$

The variable belief updates and message belief updates are repeated until the algorithm achieves convergence. In this case, convergence is reached when the sum in the absolute value of changes across all messages is less than the threshold, which was set at the value of $1 * 10^{-8}$ for the purpose of this work. This convergence criteria is checked after each message update.

5.4 Computing Node Weights

In theory, the node weights used are flexible, and can be changed depending on the problem definition. For the sake of haplotype assembly on the pCompass factor graph, I define a weighting process based on the read evidence on each node in the pCompass graph, with two additional, slight variations. First, the node weights, $w(V)$ are initialized to the total amount of read evidence supporting the corresponding node's factor. Specifically, the node weight for node $v_{i,j}$, corresponding to SNPs s_i, s_j , is set to the sum of the factors at node $v_{i,j}$, which is equal to the number of reads covering SNPs s_i, s_j .

$$w(v_{i,j}) = \sum \psi_{v_{i,j}} \quad (5.7)$$

Then, the following steps are taken to dynamically update the weights for every iteration of belief propagation.

5.4 COMPUTING NODE WEIGHTS

(1) Divide the node weights by a normalizing value. The choice for this value dictates the variation for weight updates mentioned above.

(a) In the standard case, the node weights are divided by 1, leaving the values unchanged.

(b) Alternatively, the node weights are divided by the maximum node weight, bounding all node weights between 0 and 1.

$$w(V) = w(V)/\max(w(V)) \quad (5.8)$$

(c) In the final variation examined, the node weights are divided by the mean node weight, allowing for some node weights to be greater than 1.

$$w(V) = w(V)/\text{mean}(w(V)) \quad (5.9)$$

(2) Next, the computed node weights are applied to all messages. For each message $m_{a,b}$, representing the message sent from vertex v_a to vertex v_b , the message is updated by scaling it by the corresponding weight of vertex a , $w(v_a)$.

$$m_{a,b} = w(v_a)m_{a,b} \quad (5.10)$$

(3) The beliefs are updated using the computed message updates, and then the new messages are computed based on the updated beliefs. Finally, we update the node weights by averaging the existing node weight and all incoming message weights. For vertex v_a , receiving messages from vertexes v_b , and v_c , vertex v_a 's new weight becomes an average of the weights of all three vertices.

$$w'(v_a) = \text{mean}(w(v_a), w(v_b), w(v_c)) \quad (5.11)$$

These steps are repeated within the belief propagation algorithm until convergence.

5.5 Sampling Completed Haplotypes

Once belief propagation has converged and the node and pairwise beliefs have been computed, we then sample completed haplotypes from the pCompass graph. The sampling step of the algorithm iteratively builds a completed haplotype phasing, one SNP at a time. The pCompass graph is first transformed into a directed graph, such that nodes containing SNPs earlier in the chromosome have their edges directed towards nodes containing SNPs later in the chromosome. Then, nodes are considered in topological ordering. The first node has its state, or phasing, sampled based on its computed variable belief. This state becomes fixed, and all subsequent samples are conditioned on this fixed state. The next nodes are sampled based on their variable beliefs and the pairwise beliefs for incoming edges. If an incompatible phasing is sampled, then the phasing is resampled, and the algorithm continues. This sampling process continues until all nodes have been sampled, and k complete haplotypes have been computed.

Chapter 6

Results

6.1 Motivating Example

I applied weighted sparse matrix belief propagation to the original motivating example (Figure 6.1).

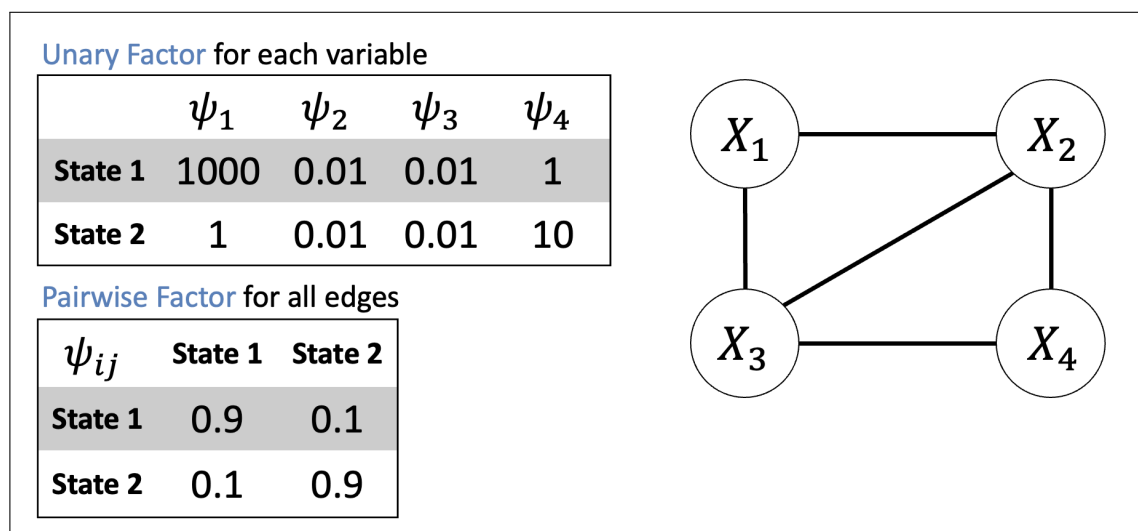


Figure 6.1: Consider this factor graph with four binary random variables. Unary factors for each variable, and the pairwise factor for all edge factors are shown in the tables on the left.

The desired behavior for inference on this graph is to see the phasing at X_1 influence the phasings in the rest of the graph more than any other node. Classical belief propagation does not achieve this behavior. I produced resulting marginal probabilities for both the

6.2 SIMULATED NA12878 DATA

standard-baseline variant of wSMBP, where the variable weight vector is left unchanged, and the max-baseline variant of wSMBP, where the variable weight vector is divided by its mean (Figure 6.2).

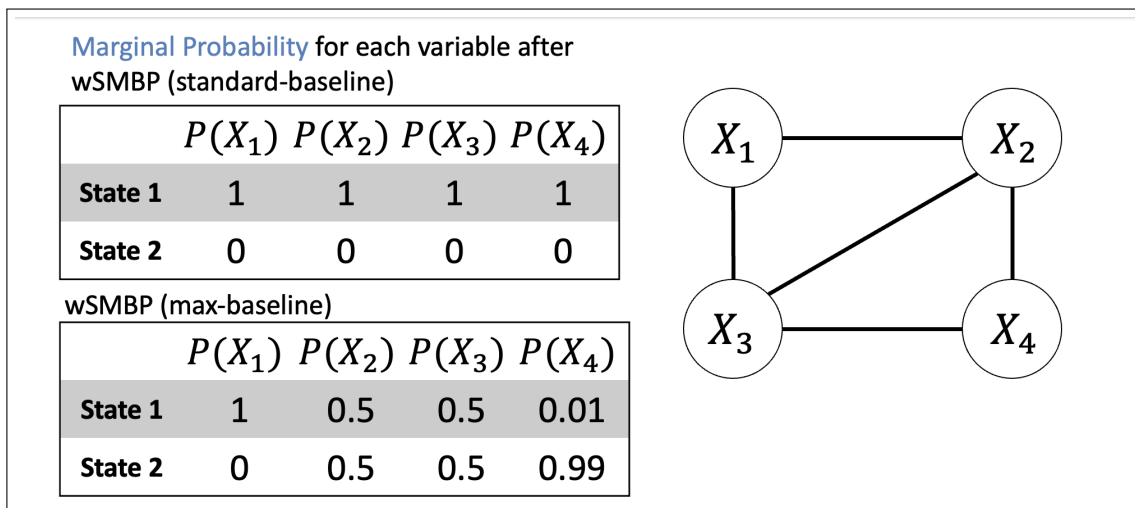


Figure 6.2: See marginal probabilities for wSMBP on the factor graph in the tables on the left.

Max-baseline wSMBP computes the same marginal probabilities for each random variable as classical belief propagation. Standard-baseline wSMBP, on the other hand, places all variables deterministically in state 1. This removal of uncertainty in the factor graph contradicts the original objective of the pHapCompass model, which is to preserve uncertainty in the model, and report that uncertainty in the end through sampling.

6.2 Simulated NA12878 Data

We simulated synthetic, virtual polyploid genomes from a sequenced human genome, NA12878. To produce a genome of ploidy k , we simulate k haplotypes based on this NA12878 genome. The ploidy values examined were $K \in \{3, 4, 6, 8\}$. We simulate two sets of contigs containing 100 and 1000 SNPs each. For each SNP position in the genome, we uniformly select a genotype, g , between 1 and $k - 1$. For each SNP position, we randomly assign the alternate allele to g haplotypes - the other haplotypes retain the reference allele. We save VCF files for

6.3 QUANTITATIVE RESULTS ON SIMULATED NA12878 DATA

each phased genotype, and generate a FASTA file for each of the k haplotypes.

Then, we use ART to simulate Illumina paired-end short reads from these genomes [6]. We simulate HiSeq 2500 sequencing technology, with the mean length of reads equal to 150 base pairs. The mean and standard deviation of insert length in these simulated reads were 300 and 30, respectively. We examined five different desired coverage values of $c \in \{10, 30, 50, 70, 100\}$.

6.3 Quantitative Results on Simulated NA12878 Data

I collected results for unweighted sparse matrix belief propagation, standard weighted sparse matrix belief propagation, and also for weighted sparse matrix belief propagation using mean weight and max weight as baselines on the simulated NA12878 data. I ran these algorithms for 100 different samples for each configuration of contig length, ploidy, read coverage, and algorithm variant.

6.3.1 Vector Error Rate

The vector error rate for a predicted haplotype-resolved assembly requires the true haplotypes present to compute. A switch operation involves splitting two or more predicted haplotypes at the same SNP position and exchanging and reassembling the fragmented haplotypes at that SNP position to match the true phasings. The vector error counts the minimum quantity of these switch operations needed in order to perfectly match the true haplotype phasings. The vector error *rate* simply divides the vector error value by the length of SNPs assembled.

I computed the vector error rate for the predicted haplotypes from each variant of SMBP. There are 100 samples for each (k, c) configuration, and for each configuration, I average the vector error rate across all 100 samples (refer to Figure 6.3). These results show that there is no variant of the method which clearly outperforms the others. As expected, performance generally improves with improving coverage, and improves at lower ploidies.

Next, I examined the performance of wSMBP using the max-baseline against performing

6.3 QUANTITATIVE RESULTS ON SIMULATED NA12878 DATA

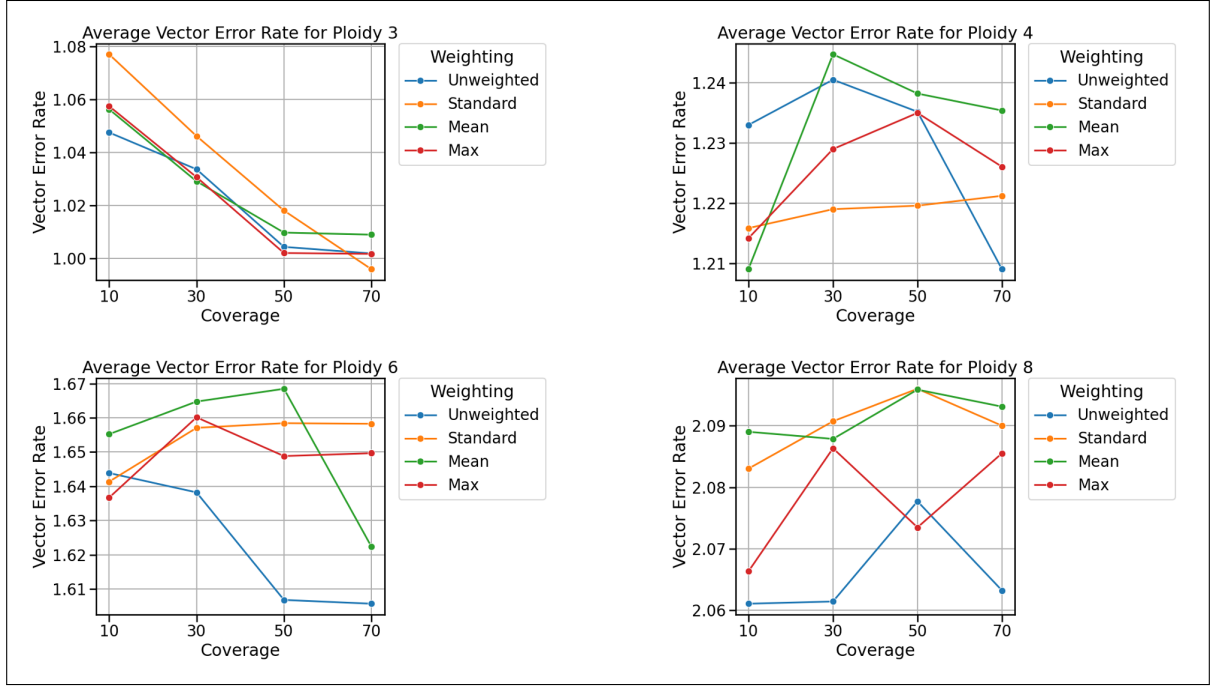


Figure 6.3: Vector error rate results for unweighted SMBP and weighted variants of SMBP

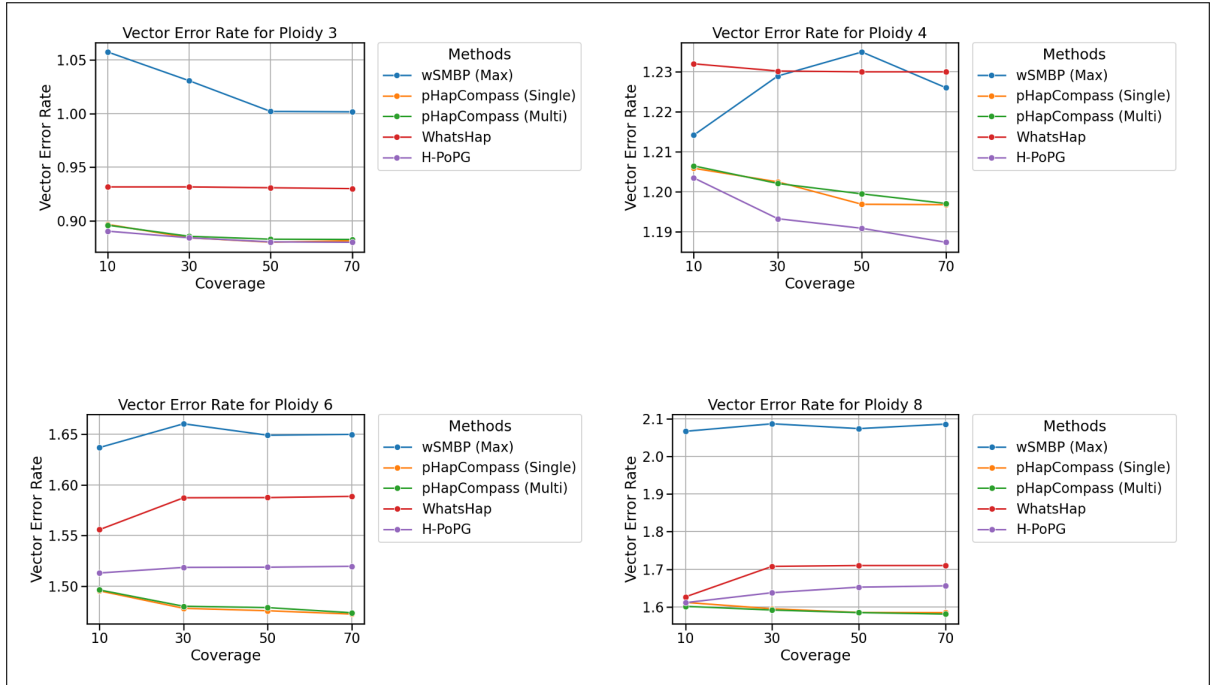


Figure 6.4: Vector error rate results for wSMBP (max-baseline), FFBS on pCompass graph, WhatsHap, and H-PoPG

forward-filtering backward-sampling on the pCompass graph ¹, as well as against H-PoPG

¹The results for FFBS on the pCompass graph are based on preliminary work not yet published

6.3 QUANTITATIVE RESULTS ON SIMULATED NA12878 DATA

and WhatsHap [11] [17]. Once again, the results for vector error rate were averaged over 100 samples for each (k, c) (refer to Figure 6.4). When comparing to other methods, in almost every setting, wSMBP is outperformed. It only remains competitive with WhatsHap in ploidy 4 for vector error rate, likely an artifact of stochasticity.

6.3.2 Pairwise Phasing Accuracy

The pairwise phasing accuracy metric also requires the true haplotypes to compute. The pairwise phasing accuracy is computed as the percentage of samples from nodes in the pCompass graph which produce the same phasing as is seen in the true haplotypes. This is not a direct measure of how well the method solves the haplotype assembly problem, it is rather a measure of how well the marginal probabilities are computed in the pCompass graph. As a result, this measure is only available to methods operated on the pCompass graph.

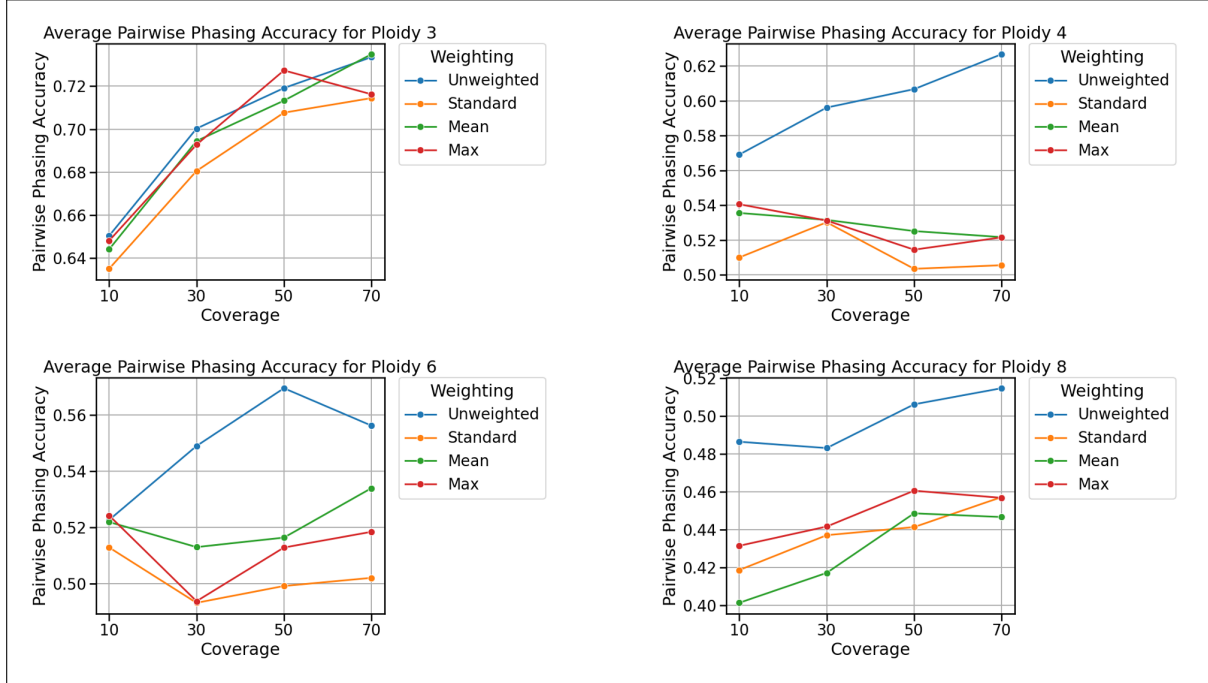


Figure 6.5: Pairwise phasing accuracy results for unweighted SMBP and weighted variants of wSMBP

I produced results for pairwise phasing accuracy to compare the marginal probabilities calculated by unweighted SMBP as well as all three variants of wSMBP (refer to Figure 6.5).

6.3 QUANTITATIVE RESULTS ON SIMULATED NA12878 DATA

Once again, results were taken from 100 samples at each (k, c) configuration and averaged. The results for pairwise phasing accuracy now show significantly better performance from unweighted SMBP when compared to the weighted versions of the algorithm, in all cases other than triploid samples.

Chapter 7

Discussion

All results on the NA12878 data, including vector error rate, minimum error correction, and pairwise phasing accuracy, suggest that all methods of weighted sparse matrix belief propagation offer no significant advantage over standard sparse matrix belief propagation, for all configurations of ploidy and read coverage. Furthermore, all variations of sparse matrix belief propagation are outperformed by state of the art methods, especially WhatsHap.

I implemented the node weighting process in belief propagation with the intention to increase the impact that informative nodes have on the beliefs in the graph. This was founded on two major assumptions. First, that there exist areas of the chromosome with higher relative coverage, and that these regions are more informative than others. Second, that information from potentially more informative nodes will be successfully passed through the graph with weighting. I believe that the first assumption was not necessarily true in this case of haplotype assembly. I suspect that most nodes are actually relatively equally informative, and that there is not as much variance in read coverage as I had suspected. Especially in simulated datasets on ideal genomes with low read error rates and high SNP density, there may actually be low variance in read coverage.

It remains interesting that belief propagation itself also performs weakly when compared to other methods. The pCompass graph itself is not an issue, as performing forward-filtering

backwards-sampling directly on the pCompass graph is at the very least competitive with, and in many cases outperforms, other state of the art methods. The pCompass graph is dense, however, and contains many cliques and cycles. I believe this presents a problem when operating belief propagation on the graph. There are likely many self-reinforcing loops which can produce counter-productive messages which may decrease performance.

I still believe dynamic weighted belief propagation to hold potential as an inference method for unique circumstances. One experiment to try to prove the utility of dynamic weighted belief propagation may be to artificially produce noise and variance in the read coverage in simulated genome data, such that some nodes in the pCompass graph certainly are more informative than others. This may produce an environment in which weighting the nodes will offer much more value. An additional experiment which may prove useful is to introduce new variants to the algorithm. The algorithm is highly modular. The sparse matrix belief propagation operations can be swapped with classical belief propagation message updates. The dynamic weighting updates could also be computed differently. There may be other measures for certainty or information on a node, such as a function of the entropies of the node factor distributions.

Chapter 8

Conclusion

In this thesis, I have proposed a variant of belief propagation, which combines sparse matrix operations to improve runtime with a novel dynamic weighting approach to scale the messages passed in the algorithm. This method aims to address a potential challenge in our probabilistic graphical representation of SNPs and DNA sequencing reads as a model for haplotype assembly, in which certain parts of the genome may contain less reads, and in turn, provide less information towards the complete haplotype phasing.

While the method did not achieve desirable results, I believe the research done has provided valuable information to the haplotype assembly problem and towards improving belief propagation as an algorithm. This algorithm provides a modular platform for experimenting with new weighting procedures and other variants of belief propagation, which may further improve performance. There may also exist other areas in which a dynamic weighted approach may prove more useful, where some nodes in a factor graph may be more informative than others. I believe there are more experiments to explore with dynamic weighted belief propagation, and could provide value to any field which utilizes probabilistic graphical models.

Bibliography

- [1] D. Aguiar and S. Istrail. “HapCompass: a fast cycle basis algorithm for accurate haplotype assembly of sequence data”. *Journal of Computational Biology* 19 (2012), 577–590.
- [2] V. Bansal and V. Bafna. “HapCUT: an efficient and accurate algorithm for the haplotype assembly problem”. *Bioinformatics* 24 (Aug. 2008), i153–i159.
- [3] R. M. Bixler. “Sparse matrix belief propagation”. PhD thesis. Virginia Tech, 2018.
- [4] S. Consul, Z. Ke, and H. Vikalo. “XHap: haplotype assembly using long-distance read correlations learned by transformers”. *Bioinformatics Advances* 3 (2023), vbad169.
- [5] B. V. Halldórsson, V. Bafna, N. Edwards, R. Lippert, S. Yooseph, and S. Istrail. “A survey of computational methods for determining haplotypes”. *RECOMB Workshop on Computational Methods for SNPs and Haplotype Inference*. Springer. 2002, 26–47.
- [6] W. Huang, L. Li, J. R. Myers, and G. T. Marth. “ART: a next-generation sequencing read simulator”. *Bioinformatics* 28 (2012), 593–594.
- [7] H.-A. Loeliger. “An introduction to factor graphs”. *IEEE Signal Processing Magazine* 21 (Jan. 2004). Conference Name: IEEE Signal Processing Magazine, 28–41.
- [8] G. A. Logsdon, M. R. Vollger, and E. E. Eichler. “Long-read human genome sequencing and its applications”. *Nature Reviews Genetics* 21 (Oct. 2020), 597–614.
- [9] J. Luo, J. Wang, J. Wei, C. Yan, and H. Luo. “DeepHapNet: a haplotype assembly method based on RetNet and deep spectral clustering”. *Briefings in Bioinformatics* 26 (2024).
- [10] K. Murphy, Y. Weiss, and M. I. Jordan. “Loopy belief propagation for approximate inference: An empirical study”. *arXiv preprint arXiv:1301.6725* (2013).
- [11] M. Patterson, T. Marschall, N. Pisanti, L. Van Iersel, L. Stougie, G. W. Klau, and A. Schönhuth. “WhatsHap: weighted haplotype assembly for future-generation sequencing reads”. *Journal of Computational Biology* 22 (2015), 498–509.
- [12] J. Pearl. “Probabilistic Reasoning in Intelligent Systems; Network of Plausible Inference”. *Morgan Kaufmann, 1988* (1988).
- [13] R. Schwartz. “Theory and algorithms for the haplotype assembly problem”. *Communications in Information and Systems* 10 (2010), 23–38.

BIBLIOGRAPHY

- [14] C. Song, S. Liu, J. Xiao, W. He, Y. Zhou, Q. Qin, C. Zhang, and Y. Liu. “Polyploid organisms”. *Science China Life Sciences* 55 (2012), 301–311.
- [15] R. Tewhey, V. Bansal, A. Torkamani, E. J. Topol, and N. J. Schork. “The importance of phase information for human genomics”. *Nature Reviews Genetics* 12 (Mar. 2011), 215–223.
- [16] T. Xiao and W. Zhou. “The third generation sequencing: the advanced approach to genetic diseases.” eng. *Translational pediatrics* 9 (Apr. 2020). Place: China, 163–173.
- [17] M. Xie, Q. Wu, J. Wang, and T. Jiang. “H-PoP and H-PoPG: heuristic partitioning algorithms for single individual haplotyping of polyploids”. *Bioinformatics* 32 (2016), 3735–3744.